```java
// FILE: C:\Users\fabio\IdeaProjects\sem2-pi-24.25-g031-repo\src\main\java\pt\ipp\isep\dei\Mdisc\controller\AppController.java
package pt.ipp.isep.dei.Mdisc.controller;

import pt.ipp.isep.dei.Mdisc.domain.Station;
import pt.ipp.isep.dei.Mdisc.domain.Line;
import pt.ipp.isep.dei.Mdisc.repo.StationRepository;
import pt.ipp.isep.dei.Mdisc.repo.LineRepository;
import pt.ipp.isep.dei.Mdisc.repo.MatrixRepository;
import pt.ipp.isep.dei.Mdisc.ui.MenuUI;
import pt.ipp.isep.dei.Mdisc.ui.GraphUI;

import java.io.PrintWriter;
import java.util.List;
import java.util.ArrayList;
import java.util.Scanner;

/**
 * Main application controller handling core computations for US13 and US14.
 * <p>
 * Responsibilities:
 * <ul>
 *   <li>Load stations and lines (with optional electrification filter).</li>
 *   <li>Export adjacency matrix (counts of parallel edges) to text file using primitive loops.</li>
 *   <li>Compute transitive closure (reachability) via triple nested loops, save result.</li>
 *   <li>Determine Eulerian feasibility (connectedness and oddâ€‰degree count).</li>
 * </ul>
 */
public class AppController {
    private final StationRepository stationRepo;
    private final String linesFile;
    private final MatrixRepository matrixRepo = new MatrixRepository();
    private boolean onlyElectrified;

    /**
     * @param stationsFile    CSV path for station definitions
     * @param linesFile       CSV path for line definitions
     * @param onlyElectrified if true, include only electrified lines in computations
     */
    public AppController(String stationsFile, String linesFile, boolean onlyElectrified) {
        this.stationRepo     = new StationRepository(stationsFile);
        this.linesFile       = linesFile;
        this.onlyElectrified = onlyElectrified;
    }

    public void start() {
        Scanner sc = new Scanner(System.in);
        int choice;
        do {
            MenuUI menu = new MenuUI(onlyElectrified);
            choice = menu.showMenu();
            LineRepository lineRepo = new LineRepository(linesFile, onlyElectrified);

            switch (choice) {
                case 1:
                    exportMatrix(lineRepo);
                    break;
                case 2:
                    checkReachabilityAndSave(sc, lineRepo);
                    break;
                case 3:
                    showFullGraph(lineRepo);
                    break;
```

```
            case 4:
                checkMaintenanceRoute(lineRepo);
                break;
            case 0:
                System.out.println("Exiting...");
                break;
            default:
                System.out.println("Invalid option");
                break;
        }
    } while (choice != 0);
    sc.close();
}

/**
 * US13 Core: Exports integer adjacency matrix to "matrix.txt".
 * <p>
 * Algorithm:
 * <ol>
 *   <li>Load stations list and lines filtered by electrification.</li>
 *   <li>Invoke {@link MatrixRepository#buildAdjacency} to build NÃ—N matrix m.</li>
 *   <li>Determine column width w based on max station name length and digit count.</li>
 *   <li>Print header row with station identifiers.</li>
 *   <li>For each row i, print station i then m[i][j] for j=0..N-1.</li>
 *   <li>All loops are primitive; formatting done with String.format.</li>
 * </ol>
 */
private void exportMatrix(LineRepository lineRepo) {
    try {
        List<Station> stations = stationRepo.loadAll();
        List<Line> lines       = lineRepo.loadAll(stations);
        int[][] m = matrixRepo.buildAdjacency(stations, lines);
        int n = stations.size();

        int maxName = stations.stream()
                .mapToInt(s -> s.toString().length())
                .max().orElse(4);
        int maxVal  = 0;
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                maxVal = Math.max(maxVal, String.valueOf(m[i][j]).length());
        int w = Math.max(maxName, maxVal) + 2;
        String nameFmt = "%" + w + "s";
        String numFmt  = "%" + w + "d";

        try (PrintWriter out = new PrintWriter("matrix.txt")) {
            out.printf(nameFmt, "");
            for (Station s : stations) out.printf(nameFmt, s.toString());
            out.println();
            for (int i = 0; i < n; i++) {
                out.printf(nameFmt, stations.get(i).toString());
                for (int j = 0; j < n; j++)
                    out.printf(numFmt, m[i][j]);
                out.println();
            }
            System.out.println("Matrix exported to matrix.txt");
        }
    } catch (Exception ex) {
        System.err.println("Error exporting matrix: " + ex.getMessage());
    }
}

/**
```

```
 * US13 Core: Computes reachability closure and saves to "closure.txt".
 * <p>
 * Algorithm (transitive closure):
 * <ol>
 *   <li>Load stations and filtered lines; build adjacency matrix m.</li>
 *   <li>Initialize boolean matrix r of size NÃ—N:
 *       <ul>
 *         <li>r[i][i] = true</li>
 *         <li>r[i][j] = (m[i][j] &gt; 0)</li>
 *       </ul>
 *   </li>
 *   <li>For k = 0 to N-1:
 *       <ul>
 *         <li>For i = 0 to N-1:
 *             <ul>
 *               <li>If r[i][k] then for j=0..N-1: r[i][j] |= r[k][j]</li>
 *             </ul></li>
 *       </ul>
 *   </li>
 *   <li>Write r matrix as 0/1 grid to "closure.txt" with row/column indices.</li>
 *   <li>Report r[src][dst] to console.</li>
 * </ol>
 * @param sc       user input scanner for selecting src and dst indices
 * @param lineRepo repository to load lines
 */
private void checkReachabilityAndSave(Scanner sc, LineRepository lineRepo) {
    try {
        List<Station> stations = stationRepo.loadAll();
        List<Line> lines       = lineRepo.loadAll(stations);
        int n = stations.size();

        for (int i = 0; i < n; i++) {
            System.out.printf("%2d) %s%n", i, stations.get(i));
        }
        System.out.print("Origin: ");
        int src = sc.nextInt();
        System.out.print("Destination: ");
        int dst = sc.nextInt();
        sc.nextLine();

        if (src < 0 || src >= n || dst < 0 || dst >= n) {
            System.out.println("Invalid index.");
            return;
        }

        int[][] m = matrixRepo.buildAdjacency(stations, lines);
        boolean[][] r = new boolean[n][n];
        for (int i = 0; i < n; i++) {
            r[i][i] = true;
            for (int j = 0; j < n; j++) {
                r[i][j] = m[i][j] > 0;
            }
        }

        for (int k = 0; k < n; k++) {
            for (int i = 0; i < n; i++) {
                if (r[i][k]) {
                    for (int j = 0; j < n; j++) {
                        r[i][j] |= r[k][j];
                    }
                }
            }
        }
```

```
        try (PrintWriter out = new PrintWriter("closure.txt")) {
            out.printf("%4s", "");
            for (int i = 0; i < n; i++) out.printf("%4d", i);
            out.println();
            for (int i = 0; i < n; i++) {
                out.printf("%4d", i);
                for (int j = 0; j < n; j++)
                    out.printf("%4d", r[i][j] ? 1 : 0);
                out.println();
            }
            System.out.println("Transitive closure saved to closure.txt");
        }

        if (r[src][dst]) {
            System.out.println("A path exists between " + stations.get(src) + " and " + stations.get(dst));
        } else {
            System.out.println("No path exists between " + stations.get(src) + " and " + stations.get(dst));
        }
    } catch (Exception ex) {
        System.err.println("Error computing reachability: " + ex.getMessage());
    }
}

private void showFullGraph(LineRepository lineRepo) {
    try {
        List<Station> stations = stationRepo.loadAll();
        List<Line> lines       = lineRepo.loadAll(stations);
        int[][] m  = matrixRepo.buildAdjacency(stations, lines);
        boolean[][] e = matrixRepo.buildElectrification(stations, lines);

        GraphUI ui = new GraphUI("FullGraph");
        ui.addStations(stations);
        ui.addEdges(stations, m, e);
        ui.display(true);

    } catch (Exception ex) {
        System.err.println("Error: " + ex.getMessage());
    }
}


/**
 * US14 Precheck: Determines if a maintenance route is possible.
 * <p>
 * Algorithm:
 * <ol>
 *   <li>Load all stations and filtered lines; build adjacency m0.</li>
 *   <li>Remove isolated verticesâ€”keep indices where sum of row>0.</li>
 *   <li>Build reduced adjacency m for kept vertices.</li>
 *   <li>Compute connectivity r via transitive closure (same as above).
 *       If any r[i][j] false, graph disconnected => no route.
 *   </li>
 *   <li>Count vertices with odd degree = count of (sum m[i][*] mod 2 != 0).
 *       If odd==0 or odd==2 => Eulerian trail/circuit exists;
 *       else => no route.
 *   </li>
 * </ol>
 * Uses only primitive loops and arithmetic.
 * @param lineRepo repository to load lines
 */
private void checkMaintenanceRoute(LineRepository lineRepo) {
    try {
```

```
        List<Station> allStations = stationRepo.loadAll();
        List<Line> lines          = lineRepo.loadAll(allStations);

        int[][] adj0 = matrixRepo.buildAdjacency(allStations, lines);
        int n0 = allStations.size();

        List<Integer> keep = new ArrayList<>();
        for (int i = 0; i < n0; i++) {
            boolean isolated = true;
            for (int j = 0; j < n0; j++) {
                if (adj0[i][j] > 0) {
                    isolated = false;
                    break;
                }
            }
            if (!isolated) keep.add(i);
        }

        List<Station> stations = new ArrayList<>();
        for (int idx : keep) stations.add(allStations.get(idx));

        int n = stations.size();
        int[][] m = new int[n][n];
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                m[i][j] = adj0[keep.get(i)][keep.get(j)];

        boolean[][] r = new boolean[n][n];
        for (int i = 0; i < n; i++) {
            r[i][i] = true;
            for (int j = 0; j < n; j++)
                if (m[i][j] > 0) r[i][j] = true;
        }
        // Warshall
        for (int k = 0; k < n; k++)
            for (int i = 0; i < n; i++)
                if (r[i][k])
                    for (int j = 0; j < n; j++)
                        r[i][j] |= r[k][j];

        boolean connected = true;
        for (int i = 0; i < n && connected; i++)
            for (int j = 0; j < n; j++)
                if (!r[i][j]) { connected = false; break; }

        int odd = 0;
        for (int i = 0; i < n; i++) {
            int deg = 0;
            for (int j = 0; j < n; j++)
                deg += m[i][j];
            if (deg % 2 != 0) odd++;
        }

        if (!connected) {
            System.out.println("Graph is disconnected â†’ maintenance route not possible.");
        } else if (odd == 0 || odd == 2) {
            System.out.printf("Graph is Eulerian (odd vertices=" + odd + ") â†’ route possible.");
        } else {
            System.out.printf("Graph is not Eulerian (odd vertices=" + odd + ") â†’ route not possible.");
        }

    } catch (Exception ex) {
        System.err.println("Error checking maintenance route: " + ex.getMessage());
```

```
            }
        }
    }
```

```java
// FILE: C:\Users\fabio\IdeaProjects\sem2-pi-24.25-g031-repo\src\main\java\pt\ipp\isep\dei\Mdisc\domain\Line.java
package pt.ipp.isep.dei.Mdisc.domain;

import java.util.Objects;

/**
 * Represents an undirected connection (edge) between two {@link Station} nodes
 * in a railway graph, with an attribute indicating whether the line is electrified.
 */
public class Line {
    private final Station from;
    private final Station to;
    private final boolean electrified;

    /**
     * Constructs a new Line connecting two stations.
     *
     * @param from          the originating station (non-null)
     * @param to            the destination station (non-null)
     * @param electrified   true if this line is equipped with electrification
     * @throws NullPointerException if {@code from} or {@code to} is null
     */
    public Line(Station from, Station to, boolean electrified) {
        this.from = Objects.requireNonNull(from, "from station must not be null");
        this.to = Objects.requireNonNull(to,   "to station must not be null");
        this.electrified = electrified;
    }

    /**
     * Returns the originating station of this line.
     *
     * @return the "from" station
     */
    public Station getFrom() {
        return from;
    }

    /**
     * Returns the destination station of this line.
     *
     * @return the "to" station
     */
    public Station getTo() {
        return to;
    }

    /**
     * Indicates whether this line is electrified.
     *
     * @return true if the line is electrified, false otherwise
     */
    public boolean isElectrified() {
        return electrified;
    }

    /**
     * Returns a string representation of the Line.
     *
     * @return a string in the format from--(electrified?)-->to
     */
    @Override
    public String toString() {
```

```
        return String.format("%s --%selectrified--> %s",
                from.getName(), electrified ? "" : "non-", to.getName());
    }
}
```

```java
// FILE: C:\Users\fabio\IdeaProjects\sem2-pi-24.25-g031-repo\src\main\java\pt\ipp\isep\dei\Mdisc\domain\Station.java
package pt.ipp.isep.dei.Mdisc.domain;

import java.util.Objects;

/**
 * Represents a node (station) in a railway network graph.
 * A station has a unique name and a type (depot, station, or terminal).
 */
public class Station {
    private final String name;
    private final StationType type;

    /**
     * Constructs a new Station with the given name and type.
     *
     * @param name the unique station name (non-null, non-empty)
     * @param type the station type (depot, station, terminal)
     * @throws NullPointerException if name or type is null
     * @throws IllegalArgumentException if name is empty
     */
    public Station(String name, StationType type) {
        this.name = Objects.requireNonNull(name, "name must not be null");
        if (name.trim().isEmpty()) {
            throw new IllegalArgumentException("Station name must not be empty");
        }
        this.type = Objects.requireNonNull(type, "type must not be null");
    }

    /**
     * Returns the station's unique name.
     *
     * @return the station name
     */
    public String getName() {
        return name;
    }

    /**
     * Returns the station's type.
     *
     * @return the station type
     */
    public StationType getType() {
        return type;
    }

    /**
     * Two stations are equal if they share the same name.
     * We do not override hashCode: equality is identity-based in collections.
     *
     * @param o the other object to compare
     * @return true if o is a Station with the same name
     */
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Station)) return false;
        Station station = (Station) o;
        return name.equals(station.name);
    }
```

```
    /**
     * Returns a string representation combining type and name, e.g. "S_Dingwall".
     *
     * @return formatted station identifier
     */
    @Override
    public String toString() {
        return String.format("%s_%s", type.name(), name);
    }
}
```

```
// FILE: C:\Users\fabio\IdeaProjects\sem2-pi-24.25-g031-repo\src\main\java\pt\ipp\isep\dei\Mdisc\domain\StationType.java
package pt.ipp.isep.dei.Mdisc.domain;

/**
 * Enumeration of station types in the railway network:
 * <ul>
 *   <li>{@code S} - Station</li>
 *   <li>{@code D} - Depot</li>
 *   <li>{@code T} - Terminal</li>
 * </ul>
 */
public enum StationType {
    S,  // Station
    D,  // Depot
    T   // Terminal
}
```

```java
// FILE: C:\Users\fabio\IdeaProjects\sem2-pi-24.25-g031-repo\src\main\java\pt\ipp\isep\dei\Mdisc\Main.java
package pt.ipp.isep.dei.Mdisc;

import pt.ipp.isep.dei.Mdisc.controller.AppController;

import java.util.Scanner;

/**
 * Entry point for the Railway Network application.
 * <p>
 * This class:
 * <ol>
 *   <li>Prompts the user for station and line CSV file paths.</li>
 *   <li>Asks the user to select a train type (electric, diesel, steam).</li>
 *   <li>Configures GraphStream properties for visualization.</li>
 *   <li>Initializes and starts the {@link AppController}.</li>
 * </ol>
 */
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // 1) Read CSV file paths
        System.out.print("Path to stations CSV file: ");
        String stationsFile = sc.nextLine().trim();
        System.out.print("Path to lines CSV file:    ");
        String linesFile = sc.nextLine().trim();

        // 2) Select train type
        System.out.println("\nSelect train type:");
        System.out.println("1) Electric");
        System.out.println("2) Diesel");
        System.out.println("3) Steam");
        System.out.print("Choice: ");
        int choice = sc.nextInt();
        sc.nextLine();
        boolean onlyElectrified = (choice == 1);

        // 3) Configure GraphStream for Swing
        System.setProperty("org.graphstream.ui", "swing");
        System.setProperty(
                "org.graphstream.ui.renderer",
                "org.graphstream.ui.j2dviewer.J2DGraphRenderer"
        );

        // 4) Start the application controller
        AppController controller =
                new AppController(stationsFile, linesFile, onlyElectrified);
        controller.start();

        sc.close();
    }
}
```

```
// FILE: C:\Users\fabio\IdeaProjects\sem2-pi-24.25-g031-repo\src\main\java\pt\ipp\isep\dei\Mdisc\repo\LineRepository.java
package pt.ipp.isep.dei.Mdisc.repo;

import pt.ipp.isep.dei.Mdisc.domain.Line;
import pt.ipp.isep.dei.Mdisc.domain.Station;

import java.io.File;
import java.io.FileNotFoundException;
import java.util.*;

/**
 * Repository responsible for loading Line instances from a file.
 * <p>
 * Each line in the file is expected to have the format:
 * <pre>
 *   stationA;stationB;electrifiedFlag;distance
 * </pre>
 * where:
 * <ul>
 *   <li><b>stationA</b> and <b>stationB</b> are station identifiers matching those in the provided station list,</li>
 *   <li><b>electrifiedFlag</b> is "1" for electrified lines or "0" otherwise,</li>
 *   <li><b>distance</b> is the positive real-valued distance between the stations.</li>
 * </ul>
 * Only lines matching the {@code onlyElectrified} filter will be returned.
 */
public class LineRepository {
    private final String linesFile;
    private final boolean onlyElectrified;

    /**
     * Constructs a LineRepository.
     *
     * @param linesFile        path to the lines definition file
     * @param onlyElectrified  if true, only electrified lines are loaded
     */
    public LineRepository(String linesFile, boolean onlyElectrified) {
        this.linesFile = Objects.requireNonNull(linesFile, "linesFile must not be null");
        this.onlyElectrified = onlyElectrified;
    }

    /**
     * Loads all Line objects by reading the configured file.
     *
     * @param stations  list of known stations (used to map names to Station instances)
     * @return list of Line instances matching the filter
     * @throws FileNotFoundException if the lines file cannot be read
     */
    public List<Line> loadAll(List<Station> stations) throws FileNotFoundException {
        // Map station name -> Station for quick lookup
        Map<String, Station> map = new HashMap<>();
        for (Station s : stations) {
            map.put(s.getName(), s);
        }

        List<Line> lines = new ArrayList<>();
        try (Scanner sc = new Scanner(new File(linesFile))) {
            while (sc.hasNextLine()) {
                String[] p = sc.nextLine().split(";");
                if (p.length < 3) {
                    continue;  // skip malformed lines
                }
                boolean electrified = p[2].trim().equals("1");
```

```
                if (onlyElectrified && !electrified) {
                    continue;
                }
                String fromName = p[0].split("_", 2)[1];
                String toName   = p[1].split("_", 2)[1];
                Station from = map.get(fromName);
                Station to   = map.get(toName);
                if (from != null && to != null) {
                    lines.add(new Line(from, to, electrified));
                }
            }
        }
        return lines;
    }
}
```

```java
// FILE: C:\Users\fabio\IdeaProjects\sem2-pi-24.25-g031-repo\src\main\java\pt\ipp\isep\dei\Mdisc\repo\MatrixRepository.java
package pt.ipp.isep.dei.Mdisc.repo;

import pt.ipp.isep.dei.Mdisc.domain.Line;
import pt.ipp.isep.dei.Mdisc.domain.Station;

import java.util.List;

/**
 * Repository building adjacency matrices for a set of stations and lines.
 * <p>
 * Provides methods to construct:
 * <ul>
 *   <li>An integer adjacency matrix counting parallel connections between stations,</li>
 *   <li>A boolean matrix indicating which station pairs are connected by electrified lines.</li>
 * </ul>
 */
public class MatrixRepository {
    /**
     * Builds an undirected adjacency matrix counting direct connections.
     * <p>
     * The returned matrix has size NÃ—N where N = stations.size().
     * Each element matrix[i][j] is the number of lines connecting
     * stations at index i and j in the provided list. The matrix is symmetric
     * and its diagonal entries remain zero (no self-loops).
     * <p>
     * Only primitive loops and array operations are usedâ€”no library graph methods.
     *
     * @param stations ordered list of stations defining the matrix indices
     * @param lines    list of lines connecting pairs of stations
     * @return integer adjacency matrix of connection counts
     */
    public int[][] buildAdjacency(List<Station> stations, List<Line> lines) {
        int n = stations.size();
        int[][] matrix = new int[n][n];
        for (Line l : lines) {
            int i = stations.indexOf(l.getFrom());
            int j = stations.indexOf(l.getTo());
            matrix[i][j]++;
            matrix[j][i]++;
        }
        return matrix;
    }

    /**
     * Builds an undirected boolean matrix indicating electrified connections.
     * <p>
     * The returned matrix has size NÃ—N where N = stations.size().
     * matrix[i][j] is true if there is at least one electrified line
     * between stations i and j. The matrix is symmetric and diagonal entries
     * remain false (no self-connections).
     * <p>
     * Only primitive loops and array operations are usedâ€”no library graph methods.
     *
     * @param stations ordered list of stations defining the matrix indices
     * @param lines    list of lines connecting pairs of stations
     * @return boolean matrix marking electrified connections
     */
    public boolean[][] buildElectrification(List<Station> stations, List<Line> lines) {
        int n = stations.size();
        boolean[][] matrix = new boolean[n][n];
        for (Line l : lines) {
```

```
            if (l.isElectrified()) {
                int i = stations.indexOf(l.getFrom());
                int j = stations.indexOf(l.getTo());
                matrix[i][j] = true;
                matrix[j][i] = true;
            }
        }
        return matrix;
    }
}
```

```java
// FILE: C:\Users\fabio\IdeaProjects\sem2-pi-24.25-g031-repo\src\main\java\pt\ipp\isep\dei\Mdisc\repo\StationRepository.java
// src/main/java/org/example/repo/StationRepository.java
package pt.ipp.isep.dei.Mdisc.repo;

import pt.ipp.isep.dei.Mdisc.domain.Station;
import pt.ipp.isep.dei.Mdisc.domain.StationType;

import java.io.File;
import java.io.FileNotFoundException;
import java.util.*;

/**
 * Repository for loading {@link Station} instances from a CSV file.
 * <p>
 * Expects a single line of input where each token is formatted as:
 * <pre>
 *   TYPE_name
 * </pre>
 * (e.g., "S_Dingwall;D_Helmsdale;T_Inverness").
 * Tokens are separated by semicolons. The repository parses each token,
 * creates a {@link Station}, and returns a sorted list by type then name.
 */
public class StationRepository {
    private final String stationsFile;

    /**
     * @param stationsFile path to the stations definition file
     */
    public StationRepository(String stationsFile) {
        this.stationsFile = Objects.requireNonNull(stationsFile, "stationsFile must not be null");
    }

    /**
     * Loads and returns all Station instances from the configured file.
     * <p>
     * Parsing logic:
     * <ol>
     *   <li>Read the first line of the file (if any).</li>
     *   <li>Split line at semicolons into tokens.</li>
     *   <li>For each token, split at the first underscore into type code and name.</li>
     *   <li>Create a {@link StationType} from the code and a new {@link Station}.</li>
     *   <li>Ignore malformed tokens.</li>
     *   <li>Sort the resulting list by station type (ordinal) then name.</li>
     * </ol>
     *
     * @return sorted List of Station instances, empty if file is empty
     * @throws FileNotFoundException if the stations file cannot be opened
     */
    public List<Station> loadAll() throws FileNotFoundException {
        List<Station> stations = new ArrayList<>();
        try (Scanner sc = new Scanner(new File(stationsFile))) {
            if (!sc.hasNextLine()) {
                return stations;
            }
            String line = sc.nextLine();
            for (String tok : line.split(";")) {
                String[] p = tok.split("_", 2);
                if (p.length == 2) {
                    StationType type = StationType.valueOf(p[0]);
                    stations.add(new Station(p[1], type));
                }
            }
```

```
        }
        stations.sort(Comparator
                .comparing(Station::getType)
                .thenComparing(Station::getName)
        );
        return stations;
    }
}
```

```
// FILE: C:\Users\fabio\IdeaProjects\sem2-pi-24.25-g031-repo\src\main\java\pt\ipp\isep\dei\Mdisc\ui\GraphUI.java
package pt.ipp.isep.dei.Mdisc.ui;

import org.graphstream.graph.*;
import org.graphstream.graph.implementations.SingleGraph;
import org.graphstream.ui.view.Viewer;
import pt.ipp.isep.dei.Mdisc.domain.Station;

import java.util.List;

/**
 * UI for visualizing a railway network using GraphStream.
 */
public class GraphUI {
    private final Graph graph;

    public GraphUI(String id) {
        graph = new SingleGraph(id);
        graph.setAttribute("ui.stylesheet",
                "node {fill-color:orange; text-size:14;} edge {text-mode:hidden;}"
        );
    }

    /**
     * Adds each station to the graph as a node.
     * <p>
     * Node IDs are the station names; each node is labeled with {@code station.toString()}.
     *
     * @param stations list of Station instances to add as nodes
     */
    public void addStations(List<Station> stations) {
        for (Station s : stations) {
            Node n = graph.addNode(s.getName());
            n.setAttribute("ui.label", s.toString());
        }
    }

    /**
     * Adds edges between stations based on adjacency and electrification matrices.
     * <p>
     * For each pair (i, j) where {@code adj[i][j] > 0}, adds an undirected edge
     * with ID "stationI-stationJ". Electrified edges are colored blue;
     * non-electrified edges are colored red.
     *
     * @param stations      ordered list of stations corresponding to matrix indices
     * @param adj           integer adjacency matrix (counts of connections)
     * @param electrified   boolean matrix indicating which connections are electrified
     */
    public void addEdges(List<Station> stations, int[][] adj, boolean[][] electrified) {
        int n = stations.size();
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (adj[i][j] > 0) {
                    String id = stations.get(i).getName() + "-" + stations.get(j).getName();
                    Edge e = graph.addEdge(id,
                            stations.get(i).getName(),
                            stations.get(j).getName());
                    if (electrified[i][j]) {
                        e.setAttribute("ui.style", "fill-color:blue;");
                    } else {
                        e.setAttribute("ui.style", "fill-color:red;");
                    }
```

```
                }
            }
        }
    }

    /**
     * Displays the constructed graph in a window.
     */
    public void display(boolean autoLayout) {
        Viewer v = graph.display();
        if (autoLayout) {
            v.enableAutoLayout();
        }
    }
}
```

```java
// FILE: C:\Users\fabio\IdeaProjects\sem2-pi-24.25-g031-repo\src\main\java\pt\ipp\isep\dei\Mdisc\ui\MenuUI.java
package pt.ipp.isep.dei.Mdisc.ui;

import java.util.Scanner;

/**
 * Console-based menu for the railway network application.
 * <p>
 * Displays options to the user and reads their selection.
 */
public class MenuUI {
    private final boolean onlyElectrified;
    private final Scanner scanner = new Scanner(System.in);

    /**
     * Constructs a new MenuUI.
     *
     * @param onlyElectrified if true, show that only electrified lines are considered
     */
    public MenuUI(boolean onlyElectrified) {
        this.onlyElectrified = onlyElectrified;
    }

    /**
     * Shows the main menu and returns the user's choice.
     *
     * @return the selected option number
     */
    public int showMenu() {
        System.out.println();
        System.out.println("========= MAIN MENU =========");
        if (onlyElectrified) {
            System.out.println("| Electrified only: YES           |");
        } else {
            System.out.println("| Electrified only: NO            |");
        }
        System.out.println("------------------------------");
        System.out.println(" 1) Export adjacency matrix to file");
        System.out.println(" 2) Check reachability Aâ†’B and save closure");
        System.out.println(" 3) Display full network graph");
        System.out.println(" 4) Find maintenance route");
        System.out.println(" 0) Exit");
        System.out.print("Select an option: ");
        return scanner.nextInt();
    }

    /**
     * Entry point for standalone testing.
     */
    public static void main(String[] args) {
        MenuUI menu = new MenuUI(false);
        int choice = menu.showMenu();
        System.out.println("You selected: " + choice);
    }
}
```